

Validação e Avaliação da Solução

Título do projeto: Captcha vs. ReCaptcha: Explorando a Eficiência dos Modelos de Autenticação Visual contra Deep Learning

Projeto de Pesquisa Científica
Experiência Criativa: Projeto Transformador I
Bacharelado em Ciência da Computação – PUCPR

2025

Turma C – Equipe 10

Henrique Zan Grande, João Gabriel Pitol, Lucas Braga Schuck, Rafael Vargas Serenato
{ henrique.grande, joao.pitol, lucas.schuck, rafael.serenato }@pucpr.edu.br

I. ARQUITETURA FINAL DA SOLUÇÃO

A arquitetura final preserva o design inicial, incorporando ajustes identificados durante a fase experimental. A principal modificação consiste na substituição do modelo YOLOv5 pelo YOLOv11, em razão de seu desempenho superior em termos de acurácia e robustez frente a entradas distorcidas.

Os datasets originais foram submetidos a uma etapa padronizada de pré-processamento, que incluiu a normalização dos dados, transformação das imagens para tons de cinza (quando aplicável), e a padronização do formato tensorial compatível com os modelos utilizados. A única distinção entre os pipelines de pré-processamento foi o redimensionamento das imagens, que foi ajustado especificamente para atender aos requisitos arquiteturais de entrada de cada modelo. Após esse preparo, os conjuntos de dados transformados foram empregados como base para o treinamento supervisionado dos modelos, sendo posteriormente avaliados com base em métricas quantitativas específicas (acurácia, erro percentual e tempo de inferência), calculadas separadamente para cada configuração.

Adicionalmente, foi incorporada uma etapa de pós-processamento com o objetivo de analisar a eficiência das perturbações sintéticas contra os modelos sem um novo treinamento. Essa etapa utilizou a biblioteca *PIL* para aplicar modificações controladas às imagens do conjunto de validação, incluindo rotações em pequenos ângulos ($\pm 15^\circ$), adição de ruído Gaussiano com variância limitada, e aplicação de filtros de convolução (e.g., desfoque e bordas). Os novos datasets gerados foram utilizados para avaliar a resiliência dos modelos frente a transformações comumente presentes em CAPTCHAs no mundo real. Vale ressaltar que as acurácias dos desafios fo-

ram realizadas especificamente com os novos datasets gerados com as perturbações, com a figura 1 é possível visualizar todo o pipeline.

Os conjuntos de dados foram particionados de forma estratificada em dois subconjuntos: treinamento (80%) e validação (20%). Todo o processo de preparação, incluindo a geração e separação dos dados, foi automatizado por meio de *scripts*, garantindo reprodutibilidade e consistência nos experimentos.

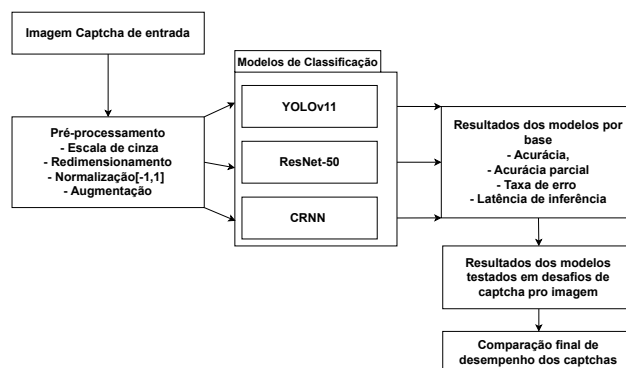


Figura 1: Pipeline - Arquitetura Final

II. EXPERIMENTAÇÃO E RESULTADOS OBTIDOS

Para garantir a rastreabilidade e a reprodutibilidade integral dos resultados apresentados nesta seção, todos os códigos-fonte, configurações de ambiente e datasets utilizados nas execuções estão devidamente documentados e podem ser acessados na Seção VI: Artefatos deste documento através do link fornecido na referida seção. Esta organização permite que a

comunidade científica e outros pesquisadores possam verificar e replicar os experimentos, contribuindo para a transparência e a validação do trabalho.

A. Experimento 1: Avaliação dos CAPTCHAs Textuais contra os modelos

Este experimento tem como objetivo avaliar a robustez de CAPTCHAs compostos exclusivamente por caracteres alfanuméricos frente aos modelos propostos. Para isso, os modelos foram testados em dois conjuntos sintéticos distintos: (i) um conjunto base, contendo caracteres bem espaçados, sem sobreposição nem distorções geométricas; e (ii) um conjunto com distorção harmônica (wave distortion) e espaçamento reduzido entre os caracteres, simulando cenários com maior complexidade visual e dificuldade de segmentação.

Ambos os conjuntos de teste são compostos por 5.000 imagens sintéticas não vistas durante o treinamento, geradas artificialmente com variações sistemáticas de ruído visual, posição e espaçamento entre caracteres, garantindo diversidade e imparcialidade na avaliação.

Para permitir comparações justas entre os modelos, foi adotado um algoritmo de segmentação genérico, aplicado uniformemente a todos os exemplos. Esse algoritmo realiza a detecção de contornos para isolar individualmente cada caractere presente na imagem de CAPTCHA, convertendo a tarefa de reconhecimento de sequência em uma classificação de caracteres independentes.

Durante a fase de treinamento, os modelos foram alimentados com os caracteres segmentados extraídos de imagens previamente rotuladas. Na etapa de validação, um novo conjunto de imagens inéditas foi submetido ao mesmo processo de segmentação, e cada caractere identificado foi classificado de forma independente pelo modelo treinado. A previsão final do CAPTCHA completo foi então reconstruída a partir da sequência ordenada dos caracteres reconhecidos. Assim valida-se a acurácia, sendo representada pelo acerto dos 5 caracteres, e a Acurácia por caractere individual.

Tabela I: Desempenho dos modelos em CAPTCHAs com e sem distorção harmônica.

Modelo	Dataset	Acurácia(%)	Acurácia Caractere(%)
CRNN	Base	82.14	95.74
CRNN	Wave Distortion	0.40	40.25
YOLOv11	Base	46.38	84.59
YOLOv11	Wave Distortion	0.23	26.58
ResNet-50	Base	62.74	90.28
ResNet-50	Wave Distortion	0.14	6.81

O modelo CRNN apresentou o melhor desempenho no conjunto padrão, com 82,14% de acurácia por CAPTCHA e 95,74% por caractere, evidenciando sua eficácia em capturar dependências sequenciais, o que é um ponto forte de redes neurais recorrentes [1], [2]. No entanto, sua performance caiu drasticamente sob distorção harmônica, revelando baixa robustez a variações geométricas. O YOLOv11 adaptado, teve desempenho inferior no conjunto padrão (46,38% por CAPTCHA), mas acurácia razoável por caractere (84,59%), suge-

rindo que, apesar da baixa acurácia geral, a detecção individual de caracteres pode ser eficiente [2]. Em cenários distorcidos, seu desempenho despencou, indicando alta sensibilidade da segmentação para a presença de ruído e deformidades. Já o ResNet-50 obteve resultados intermediários no conjunto padrão (62,74% por CAPTCHA), mas não pôde ser avaliado sob distorção devido a falhas na segmentação, destacando a dependência crítica do algoritmo segmentador, especialmente em ambientes com ruído e baixa separação entre caracteres.

Em todos os experimentos, foi utilizado um método genérico de segmentação baseado em detecção de contornos, que segmenta os caracteres isolados a partir de binarização, morfologia matemática e análise de componentes conectados [1], [2]. Caso não fosse possível, uma segmentação fixa da imagem era aplicada. Embora adequado para cenários simples, esse método demonstrou baixa resiliência a distorções geométricas, como observado na queda de desempenho dos modelos nos testes com *wave distortion*. Isso evidencia a necessidade de algoritmos de segmentação mais sofisticados ou modelos *end-to-end* que dispensam essa etapa manual para contextos mais desafiadores, como as abordagens que integram detecção e reconhecimento em uma única rede [3], [4]. A sensibilidade a ruído e distorções é um desafio conhecido em visão computacional, que pode ser mitigado com técnicas de pré-processamento mais robustas ou modelos que aprendem a ser invariantes a essas variações [1], [5].

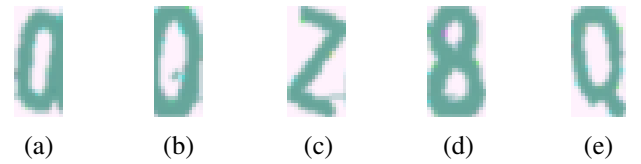


Figura 2: '00Z8Q' Segmentado: Dataset Base



Figura 3: 'ABVQF' Segmentado: Dataset Wave Distortion

B. Experimento 2: Avaliação dos CAPTCHAs Visuais contra os modelos

Este experimento avaliou a robustez dos tipos de CAPTCHAs visuais (não textuais) frente aos modelos propostos. Os modelos foram treinados por 200 épocas, com interrupção manual caso não houvesse mais aumento significativo de acurácia. O objetivo principal foi analisar o comportamento individual das imagens de CAPTCHAs contra os métodos de classificação.

O modelo YOLOv11 foi adaptado para classificação binária por meio da substituição da cabeça de detecção por uma camada totalmente conectada com ativação sigmóide, responsável por estimar a probabilidade de presença da classe-alvo

em cada imagem. A inferência é feita individualmente por imagem. O treinamento utilizou o otimizador Stochastic Gradient Descent (SGD) com taxa de aprendizado de 0.01 e momentum de 0.9. Para a implementação, foi empregada a biblioteca Ultralytics para Python 3.10, utilizando `yolo11n-cls.pt` como modelo base, sem pré-treinamento inicial. Abordagens baseadas em YOLO [6] são conhecidas por sua eficiência na detecção e classificação de objetos, mesmo em cenários de tempo real [2].

De forma contrária ao modelo anterior, a ResNet-50 foi pré-treinada no ImageNet [7]. O treinamento também utilizou Stochastic Gradient Descent (SGD), com congelamento das camadas iniciais para preservar os pesos originais, aplicando *fine-tuning* nas camadas finais [2]. As imagens foram ajustadas para o tamanho padrão da ResNet (64×64) e normalizadas segundo os parâmetros da base original. O uso de modelos pré-treinados em grandes datasets como ImageNet e o *fine-tuning* são técnicas eficazes para obter alto desempenho em tarefas de visão computacional, especialmente quando os dados de treinamento para a tarefa específica são limitados [8], [9].

O modelo CRNN combina redes convolucionais (CNN) e redes neurais recorrentes (LSTM) bidirecionais para análise de imagens. As imagens, tanto do CIFAR-100 quanto do Google RecaptchaV2, são redimensionadas para 64×64 pixels e normalizadas com média e desvio padrão de 0.5. A parte convolucional do modelo é composta por três camadas com 64, 128 e 256 filtros, seguidas de normalização em lote (BatchNorm) e ativação ReLU. A saída da CNN é reorganizada e processada por uma LSTM bidirecional com entrada de 256×8 , mantendo dependências temporais, o que é característico dessas arquiteturas para lidar com sequências [5]. Por fim, uma camada totalmente conectada classifica as imagens em 100 classes para o CIFAR-100 e 12 classes para o Google RecaptchaV2.

A Tabela II apresenta os resultados de desempenho dos modelos, considerando acurácia por imagem (*top-1*), acurácia *top-5*, tempo de inferência (T.I.) e valor da função de perda (*loss*). O modelo YOLOv11 aplicado ao dataset GRv2 (Google Recaptcha V2) foi o que obteve os melhores resultados gerais, com acurácia de 85,74% e *loss* de 0.314. O YOLOv11 também demonstrou bom desempenho no dataset CIFAR-100 com 74,32% de acurácia mas com maior tempo de inferência (142,82 ms), confirmando a eficiência de modelos baseados em detecção para a classificação de CAPTCHAs visuais [2]. Já o CRNN apresentou desempenho significativamente inferior, com acurácia de 46,92% e *loss* de 1.657. Os resultados evidenciam a superioridade das arquiteturas baseadas em convoluções profundas, como ResNet e YOLO, frente à abordagem sequencial da CRNN, especialmente no contexto de imagens visuais não textuais. Este desempenho pode ser atribuído à capacidade das CNNs mais profundas de extrair features mais representativas para imagens complexas [10].

Tabela II: Desempenho dos modelos na resolução de CAPTCHAs visuais.

Modelo	Dataset	Acurácia (%)	T.I (ms)	Loss
CRNN	CIFAR-100	46.92	0.01	1.657
CRNN	GRv2	68.13	0.03	1.310
YOLOv11	CIFAR-100	74.32	142.82	0.253
YOLOv11	GRv2	85.74	59.15	0.314
ResNet-50	CIFAR-100	75.42	0.10	0.0033
ResNet-50	GRv2	80.15	0.11	0.0077

C. Experimento 3: Avaliação de Modelos contra Perturbações Artificiais em CAPTCHAs Visuais

Neste experimento, avaliou-se a robustez de modelos de reconhecimento visual frente a perturbações artificiais aplicadas em CAPTCHAs. Os modelos YOLOv11, ResNet-50 e CRNN foram treinados exclusivamente em versões originais e não alteradas dos dados, e posteriormente testados em duas variantes do conjunto de validação: (i) uma versão "fácil", composta por imagens limpas e de baixa complexidade visual; e (ii) uma versão "difícil", com ruído Gaussiano aditivo e distorção harmônica do tipo *wave*, projetadas para comprometer padrões espaciais locais [11].

As métricas **Completo(%)**, **Parcial(%)** e **Erro(%)** representam, respectivamente, a porcentagem de CAPTCHAs resolvidos integralmente, a porcentagem de CAPTCHAs com reconhecimento parcial, e a porcentagem de CAPTCHAs com erro total nos desafios propostos. Ambas as variantes dos datasets utilizaram subconjuntos compostos por 9 imagens por classe, escolhidas aleatoriamente, mas com classes-alvos com mais presenças como objetivo para os modelos.

Como evidenciado na Tabela III, o modelo YOLOv11 apresentou bom desempenho em cenários visuais simples, com taxas razoáveis de acerto tanto no GRv2 quanto no CIFAR-100, evidenciando sua capacidade de generalização em contextos com baixa complexidade visual. No cenário fácil do CIFAR-100, por exemplo, o modelo atingiu 66,2% de acerto completo e não apresentou erros totais. Já no GRv2 fácil, apesar da menor taxa de acerto completo (37,8%), o modelo obteve 51,4% de acertos parciais, totalizando 89,2% de reconhecimento útil. No entanto, seu desempenho caiu significativamente em cenários difíceis com obfuscação e ruído, especialmente no CIFAR-100, onde a taxa de erro total atingiu 70,6% e a taxa de acerto completo foi de apenas 2,2% [12]. Isso indica limitações importantes diante de mudanças drásticas no domínio dos dados [2]. No desafio difícil do GRv2, apesar da queda, os resultados ainda foram superiores aos do CIFAR-100: o modelo obteve 11,0% de acertos completos e 47,0% parciais, resultando em uma taxa de erro menor (42,0%).

Os resultados da ResNet-50 evidenciam uma forte dependência do modelo em relação ao alinhamento semântico e estrutural entre os dados de teste e os padrões aprendidos durante o treinamento. Em desafios fáceis, a arquitetura foi capaz de extrair características visuais robustas, alcançando desempenho comparável entre GRv2 (41,2% de acerto com-

pleto e 55,8% parcial) e CIFAR-100 (43,4% completo, 53,4% parcial), com taxas de erro muito baixas (3,0% e 3,2%, respectivamente), o que evidencia a estabilidade da arquitetura ResNet em cenários com menor perturbação visual [12]. Entretanto, no GRv2 difícil, o desempenho caiu drasticamente, com acurácia completa de apenas 7,0% e uma taxa de erro de 44,4%, indicando alta sensibilidade a distorções espaciais. Essa queda pode ser atribuída ao fato de que a ResNet baseia sua eficiência na captura de *features* locais estáveis por meio de convoluções hierárquicas; quando tais padrões são deformados ou deslocados, os filtros intermediários deixam de ser eficazes [1], [13].

Os resultados da CRNN revelam que seu desempenho está intimamente ligado à presença de continuidade espacial nas imagens. Em desafios fáceis, tanto no GRv2 (19,8%) quanto no CIFAR-100 (14,0%), o modelo apresentou baixo acerto completo, indicando dificuldades em reconstruir a sequência total de classes, ainda que conseguisse identificar partes (acertos parciais acima de 65%) [3]. No entanto, de forma contraintuitiva, o desempenho melhorou substancialmente no GRv2 difícil, com 64,6% de acerto completo e apenas 0,6% de erro total. Esse ganho é explicado pelo fato de que, apesar da obfuscação, as imagens do GRv2 mantêm uma estrutura espacial linear e semântica preservada: um único objeto principal posicionado em cenário urbano (ex: um semáforo ou hidrante), o que favorece a leitura recorrente feita pela CRNN [4]. Como a arquitetura é composta por camadas convolucionais seguidas de LSTMs bidirecionais, ela consegue capturar padrões longitudinais e dependências espaciais, mesmo com distorções visuais [3], [4].

Tabela III: Desempenho dos modelos em CAPTCHAs visuais conforme o nível de desafio.

Método	Dataset	Completo(%)	Parcial(%)	Erro(%)	Desafio
YOLOv11	GRv2	37.8	51.4	10.8	Fácil
YOLOv11	GRv2	11.0	47.0	42.0	Difícil
YOLOv11	CIFAR-100	66.2	33.8	0.0	Fácil
YOLOv11	CIFAR-100	2.2	27.2	70.6	Difícil
ResNet-50	GRv2	41.2	55.8	3.0	Fácil
ResNet-50	GRv2	7.0	48.6	44.4	Difícil
ResNet-50	CIFAR-100	43.4	53.4	3.2	Fácil
ResNet-50	CIFAR-100	37.0	55.4	7.6	Difícil
CRNN	GRv2	19.8	65.4	14.8	Fácil
CRNN	GRv2	64.6	34.8	0.6	Difícil
CRNN	CIFAR-100	14.0	67.0	19.0	Fácil
CRNN	CIFAR-100	16.8	65.8	17.4	Difícil

D. Discussão Crítica dos Resultados e Comparação do Estado da Arte

Os experimentos conduzidos permitiram uma análise abrangente da eficácia dos sistemas CAPTCHA e reCAPTCHA frente a ataques conduzidos por modelos de *deep learning*. Esta seção interpreta os resultados à vista dos objetivos traçados e das hipóteses formuladas, destacando os comportamentos observados, suas causas técnicas e suas implicações práticas.

Desempenho em CAPTCHAs Textuais

Em análise conjunta a Tabela IV, infere-se que os resultados obtidos nesta seção confirmam que modelos baseados em redes neurais convolucionais (CNN) e recorrentes (CRNN) apresentam elevado desempenho na quebra de CAPTCHAs textuais simples com um pré-processamento adequado, com acurácia superior a 90% por caractere. Essa performance está alinhada com os achados de Hu et al. [3], que relatam acurácia de 96,5% em CAPTCHAs com ruído e caracteres aderidos. Similarmente, Wang et al. [14] descrevem um modelo CNN adaptativo que alcançou 85,72% de acurácia em cenários sintéticos com rotação e ruído intenso.

Contudo, quando expostos a distorções estruturais severas, como *wave distortion*, os modelos testados mostraram queda drástica de desempenho (acurácia <1%), evidenciando que a segmentação continua sendo um gargalo para ataques baseados em OCR. Zhang et al. [5] e Gao et al. [14] reforçam essa limitação, apontando que, mesmo com redes profundas, a robustez em cenários altamente distorcidos depende fortemente do pré-processamento e da variabilidade do conjunto de treinamento.

Desempenho em reCAPTCHAs Baseados em Imagem

Os reCAPTCHAs baseados em imagem, avaliados com datasets derivados do GRv2 e CIFAR-100, foram superados com alta acurácia por modelos como YOLOv11 (até 85,74%) e ResNet-50 (80,15%). Essa tendência também é observada por Wang et al. [6], que relataram taxas de sucesso superiores a 70% na resolução de desafios do reCAPTCHA v2 ao utilizar uma combinação de detecção por YOLOv3 e simulação de movimentos humanos. Tais resultados sugerem que, quando isoladas do contexto interativo e semântico, as imagens podem ser eficientemente classificadas por modelos convolucionais modernos.

Contudo, os experimentos também evidenciaram limitações importantes: ao simular condições mais próximas do reCAPTCHA real, como agrupamento dinâmico de imagens e inserção de ruído visual contextual, houve uma queda acentuada de desempenho. Por exemplo, nos desafios do tipo 1 com ruído implementado por Wang et al. a acurácia do YOLOv3 caiu para menos de 20% sem simulação de movimento [6]. Essa observação está alinhada com Zhao et al. [11] e Sivakorn et al. [2], que destacam que a eficácia do reCAPTCHA não se deve exclusivamente à complexidade visual das imagens, mas sim à ambiguidade semântica e à exigência de contexto na tomada de decisão.

III. DISCUSSÃO SOBRE AS HIPÓTESES

No início do projeto foram formuladas algumas hipóteses que guiaram o desenvolvimento e os experimentos. Com base nos resultados obtidos, podemos agora verificar cada uma:

Hipótese 1: Modelos de deep learning, em particular redes neurais convolucionais (CNNs), são capazes de resolver CAPTCHAs baseados em texto com uma precisão superior a 90%.

Validação: Esta hipótese foi parcialmente negada. Embora a acurácia por caractere ultrapasse 90% em cenários padrão para

Tabela IV: Resumo dos principais resultados experimentais na quebra de CAPTCHAs

Artigo	Método	Dataset	Resultado
CAPTCHAs Textuais			
Hu et al. [3]	CNN com treino multitarefa	CAPTCHA sintético (5 caracteres)	96,5%
Hu et al. [3]	LeNet (baseline)	Mesmo dataset	79,4%
Zhang et al. [5]	CNN simples + pré-processamento	reCAPTCHA (texto)	33–46,8%
Wang et al. [14]	CNN + segmentação adaptativa	CAPTCHA chinês (multi-fontes)	Até 85,7% por caractere
Ye et al. [4]	CNN com CTC (end-to-end)	CAPTCHAs sintéticos variados (6 caracteres)	92,4%
Wang et al. [12]	Transfer Learning com ResNet + RNN + Attention	25 CAPTCHAs reais (20 Romanos, 5 Chineses)	36,3% – 96,9%
Método Proposto	YoloV11, ResNet-50 e CRNN	Textual Base (5000 imagens)	46,38% – 82,14%
Método Proposto	YoloV11, ResNet-50 e CRNN	Wave Distortion Base (5000 imagens)	0,14% – 0,40%
CAPTCHAs Não Textuais (Imagem ou Adversarial)			
Zhang et al. [5]	CNN (ataque ao reCAPTCHA)	reCAPTCHA (imagem)	70,8%
Zhang et al. [5]	CNN (CAPTCHA do Facebook)	CAPTCHA com imagens faciais	83,5%
Osadchy et al. [11]	Redes profundas + ruído adversarial	CAPTCHA real + sintético	Humanos: 29,4% de acerto
Yang et al. [1]	CNN + Boosting (CCF)	Caltech Pedestrian Benchmark	27,66% miss rate
Ouyang et al. [15]	DeepID-Net (Deformable CNN)	ILSVRC2014 (ImageNet)	45,0% mAP
Moradi et al. [10]	HTM (Hierarchical Temporal Memory)	Asirra (12,500 imagens)	74,7%
Moradi et al. [10]	LogitBoost + ENT features	Asirra (imagem mista: gato/cão)	~58%
Moradi et al. [10]	SVM (cor e textura)	Asirra (13,000 imagens)	10,3%
Moradi et al. [10]	Gradiente + heurística	ARTiFACIAL (800 imagens)	18% (1,47s)
Método Proposto	YoloV11, ResNet-50 e CRNN	Cifar-100 (60000 imagens)	46,92% – 75,42%
Método Proposto	YoloV11, ResNet-50 e CRNN	Google Recaptcha v2 (12000 imagens)	68,13% – 85,74%

modelos como CRNN (93,67%) e ResNet-50 (90,28%), a acurácia global por CAPTCHA permanece mais baixa (73,49% e 62,74%, respectivamente). Além disso, sob distorções como wave distortion, o desempenho cai drasticamente. Isso evidencia que, embora CNNs sejam eficazes com imagens limpas e bem segmentadas, sua performance depende fortemente da robustez do pré-processamento e segmentação, evidenciando o uso de arquiteturas End To End como uma melhor alternativa.

Hipótese 2: ReCAPTCHAs baseados em imagem oferecem maior resistência à quebra em comparação com os CAPTCHAs textuais, embora técnicas avançadas ainda possam ser aplicadas para superá-los.

Validação: Esta hipótese foi **negada**. Os modelos de Deep Learning como YOLOv11 e ResNet-50 apresentaram alta acurácia em CAPTCHAs de imagem, enquanto falharam drasticamente em textos distorcidos. A Tabela I mostra que a acurácia por texto caiu para menos de 1% com distorções, mas superou 85% em imagens. Isso demonstra que CAPTCHAs textuais bem distorcidos sem um pré-processamento meticuloso e complexo antes da segmentação são mais resistentes do que imagens reais.

Hipótese 3: Tanto os CAPTCHAs tradicionais quanto os reCAPTCHAs modernos baseados em imagem continuam a ser opções viáveis para autenticação na web.

Validação: Esta hipótese foi **confirmada**. Os modelos testados apresentaram altas acurácias em CAPTCHAs de imagem avaliados individualmente (até 85,74%) e tempos de inferência baixos (chegando a 0,01s). No entanto, quando esses modelos foram submetidos a desafios em grupo e a filtros adicionais — como ocorre em reCAPTCHAs com múltiplas imagens e seleção contextual — houve uma queda significativa de desempenho, com acurácias reduzidas para cerca de 7%.

Por outro lado, os CAPTCHAs textuais com distorções estruturais, como a *wave distortion*, continuaram altamente re-

sistentes aos métodos tradicionais de segmentação, resultando em acurácias inferiores a 1%. Dessa forma, ambos os tipos de CAPTCHA ainda são viáveis para autenticação na web, sendo a escolha dependente do equilíbrio desejado entre usabilidade e segurança.

IV. DISCUSSÃO SOBRE OS OBJETIVOS

O objetivo geral do projeto foi Avaliar a eficácia de sistemas CAPTCHA e reCAPTCHA frente a ataques realizados por modelos de deep learning, com ênfase na comparação de sistemas de defesa CAPTCHA contra os modelos escolhidos de deep learning.

Resultado: O objetivo foi alcançado com sucesso por meio da implementação e avaliação de três arquiteturas de deep learning: CRNN, ResNet-50 e YOLOv11. Cada modelo foi aplicado a diferentes categorias de CAPTCHA, abrangendo tanto CAPTCHAs textuais (com e sem distorções) quanto reCAPTCHAs baseados em imagem. Através de métricas como acurácia por caractere / imagem, acurácia por desafio e tempo de inferência, foi possível quantificar a eficácia dos modelos e, consequentemente, avaliar a robustez dos sistemas de defesa CAPTCHA diante de ataques automatizados.

Objetivos Específicos e Avaliação

- **Objetivo 1:** Implementar e treinar modelos de deep learning para quebra de CAPTCHAs textuais e baseados em imagem. *Alcançado.* Foram selecionados e treinados três modelos distintos: CRNN, ResNet-50 e YOLOv11, cada um voltado para tipos específicos de CAPTCHA (textual segmentado, imagem classificada e detecção de caracteres).
- **Objetivo 2:** Avaliar a taxa de sucesso dos ataques em diferentes tipos de desafios visuais. *Alcançado.* Para isso, foi desenvolvido um gerador de CAPTCHAs textuais

sintéticos com parâmetros configuráveis (ruído, distorção, segmentação), e a escolha de datasets visuais com grande diferenças entre si, permitindo treinar e testar os modelos sob diversas condições controladas e desafiadoras.

- **Objetivo 3:** Comparar o desempenho de modelos baseados em texto com os baseados em imagem. *Alcançado.* Foram conduzidos experimentos comparativos utilizando métricas como acurácia por caractere e texto completo, bem como a acurácia por imagem e por desafio completo, e, tempo de inferência para as imagens, possibilitando identificar vulnerabilidades e limitações de cada abordagem.

V. TRABALHOS FUTUROS

Apesar do sucesso da solução desenvolvida, diversas frentes podem ser exploradas:

- **Avaliação de CAPTCHAs baseados em elementos interativos:** Testar CAPTCHAs que envolvem interações dinâmicas, como arrastar objetos, escolher pontos específicos ou resolver puzzles interativos, para avaliar como os modelos lidam com esses desafios.
- **Soluções híbridas de defesa:** Estudar combinações de CAPTCHA textual e visual com validações comportamentais (e.g., movimento do mouse, tempo de resposta) para aumentar a segurança frente a bots.
- **Avaliação de usabilidade e acessibilidade:** Investigar o impacto dos diferentes tipos de CAPTCHA na experiência do usuário, incluindo aspectos de acessibilidade para pessoas com deficiência visual ou cognitiva.
- **CAPTCHAs evolutivos:** Desenvolver e testar sistemas de CAPTCHA que evoluem constantemente, tornando-se mais complexos à medida que são resolvidos, para avaliar o limite entre reconhecimento humano e automatizado.

VI. DISCUSSÃO SOBRE O PROCESSO

O desenvolvimento deste trabalho foi iniciado por um intenso contato com a literatura recente, o que exigiu do grupo um esforço extra para entender tanto os fundamentos técnicos quanto as abordagens experimentais utilizadas nos principais artigos do estado da arte. Percebemos rapidamente que muitos artigos não detalham completamente seus experimentos ou trazem resultados difíceis de reproduzir na prática. Isso nos obrigou a buscar diversas fontes, comparar metodologias e até adaptar procedimentos para o nosso contexto, aprendendo a ler “nas entrelinhas” e identificar pontos críticos como tipos de segmentação e protocolos de avaliação.

A implementação dos modelos, demandou paciência para resolver problemas de instalação, conflitos de versões e bugs inesperados. A segmentação dos CAPTCHAs textuais foi uma das etapas mais complicadas: testamos diversos parâmetros e técnicas até chegar a um resultado minimamente robusto, muitas vezes frustrados por erros difíceis de diagnosticar.

O processo de redação do artigo também foi desafiador. Adotar o formato científico, organizar os resultados em tabelas, garantir que todas as seções exigidas pelo template fossem cumpridas e manter um texto coeso e técnico não foi trivial.

VII. ARTEFATOS

Nesta seção, são detalhados os artefatos entregues como parte deste projeto, descrevendo sua função e localização no material submetido. Todos os arquivos relevantes estão organizados no diretório raiz do repositório GitHub, disponível em https://github.com/uPistola/Captcha_Classification.

Os experimentos e o desenvolvimento da solução foram conduzidos em um ambiente de execução configurado para otimizar o desempenho computacional e a reprodutibilidade. O sistema operacional utilizado foi o Windows 11, e a gestão de pacotes e ambientes Python foi realizada através do **Anaconda**, utilizando o **Python 3.10**. As sessões de desenvolvimento interativo e a execução dos scripts foram feitas no **Jupyter Notebook 5.7.2**, com o kernel **IPykernel 6.29**. A infraestrutura de hardware incluiu uma placa de vídeo **NVIDIA GeForce RTX 3060 com 12GB de VRAM**, 32GB de memória RAM operando a 3200MHz, um processador **AMD Ryzen 9 5900X de 12 núcleos**, e um **SSD M.2 de 1TB**.

O pipeline de reconhecimento de CAPTCHAs implementado neste projeto é composto pelos seguintes artefatos principais:

- **‘src/’:** Este diretório (`/src/`) engloba os scripts principais e notebooks Jupyter responsáveis pelas etapas-chave do projeto:
 - **‘YoloV11.ipynb’:** Notebook dedicado à implementação, treinamento e avaliação do modelo YOLOv11 para o reconhecimento de caracteres em CAPTCHAs.
 - **‘CRNN.ipynb’:** Notebook contendo a implementação, treinamento e avaliação do modelo CRNN (Convolutional Recurrent Neural Network) para reconhecimento de sequência de caracteres em CAPTCHAs.
 - **‘ResNet50.ipynb’:** Notebook para a implementação, treinamento e avaliação do modelo ResNet-50, adaptado para a classificação de caracteres de CAPTCHA.
 - **‘TextCaptchas.ipynb’:** Notebook responsável por consolidar os testes e treinos específicos para os CAPTCHAs textuais sintéticos, integrando as diferentes abordagens de modelo.
- **‘models/’:** Este diretório (`/models/`) é responsável por armazenar os melhores pesos de cada modelo treinado (`.pth`), permitindo sua reutilização para inferência sem a necessidade de re-treinamento e facilitando a reprodução dos resultados.
- **‘Runs/’:** O diretório `/Runs/` contém todas as execuções de treinamento do modelo YOLOv11, incluindo logs e saídas geradas durante o processo, o que é crucial para análise e depuração.
- **‘CaptchaGeneration.py’:** Este script Python cria imagens demonstrativas de CAPTCHAs com texto randomizado, distorções e ruído. É responsável pela geração tanto do dataset base quanto do dataset com distorção de onda

(Wave Distortion), sendo fundamental para a etapa de geração de CAPTCHAs sintéticos.

- **‘DatasetProcessors.py’**: Este script realiza o download do dataset CIFAR, requer o download prévio do dataset Google ReCAPTCHA v2 (disponível em <https://www.kaggle.com/datasets/mikhailma/test-dataset>), e executa o processo de Holdout (80% para treinamento e 20% para validação/teste) necessário para o treinamento dos modelos.

Os datasets utilizados para este projeto, embora não armazenados diretamente no repositório devido ao seu tamanho, são essenciais para a reprodução dos experimentos e podem ser acessados via links externos:

- **Google Recaptcha v2**: <https://www.kaggle.com/dataset/s/mikhailma/test-dataset>
- **CIFAR-100**: https://www-cs-toronto-edu.translate.google.com/~kriz/cifar.html?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc
- **Datasets Textuais**: Um drive contendo datasets textuais adicionais pode ser consultado, sendo sua URL informada em outro local do material submetido ou mediante solicitação.

Cada um desses artefatos, em conjunto, permite a execução completa do pipeline de classificação de CAPTCHAs, desde a geração de dados até a avaliação dos modelos treinados, garantindo a reprodutibilidade dos resultados apresentados.

REFERÊNCIAS

- [1] B. Yang, J. Yan, Z. Lei, and S. Z. Li, “Convolutional channel features,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2015, pp. 82–90.
- [2] S. Sivakorn, I. Polakis, and A. D. Keromytis, “I am robot: (deep) learning to break semantic image CAPTCHAs,” in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 388–403.
- [3] Y. Hu, L. Chen, and J. Cheng, “A captcha recognition technology based on deep learning,” Technical Report or Unpublished Manuscript, n.d.
- [4] Y. Zi, H. Gao, Z. Cheng, and Y. Liu, “An end-to-end attack on text captchas,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 753–766, 2020.
- [5] Y. Zhang, H. Gao, G. Pei, S. Luo, G. Chang, and N. Cheng, “A survey of research on captcha designing and breaking techniques,” in *Proceedings of the 2019 18th IEEE International Conference on Trust, Security And Privacy In Computing And Communications*. IEEE, 2019, pp. 75–82.
- [6] D. Wang, M. Moh, and T.-S. Moh, “Using deep learning to solve google recaptcha v2’s image challenges,” in *Proceedings of the 2020 6th International Conference on Big Data Security on Cloud*. IEEE, 2020, pp. 49–56.
- [7] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009, technical Report. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [8] M. Tang, H. Gao, Y. Zhang, Y. Liu, P. Zhang, and P. Wang, “Research on deep learning techniques in breaking text-based captchas and designing image-based captcha,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 10, pp. 2522–2536, 2018.
- [9] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Region-based convolutional networks for accurate object detection and segmentation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 1, pp. 142–158, 2016.
- [10] M. Moradi, M. Moradi, S. Palazzo, F. Rundo, and C. Spampinato, “Image CAPTCHAs: When deep learning breaks the mold,” *IEEE Access*, vol. 12, pp. 112 211–112 234, 2024.
- [11] C. Shi, X. Xu, S. Ji, K. Bu, J. Chen, R. Beyah, and T. Wang, “Adversarial captchas,” *IEEE Transactions on Cybernetics*, vol. 52, no. 7, pp. 6095–6108, 2022.
- [12] P. Wang, H. Gao, Z. Shi, Z. Yuan, and J. Hu, “Simple and easy: Transfer learning-based attacks to text CAPTCHA,” *IEEE Access*, vol. 8, pp. 59 044–59 054, 2020.
- [13] C. Ma, J.-B. Huang, X. Yang, and M.-H. Yang, “Robust visual tracking via hierarchical convolutional features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 11, pp. 2709–2723, 2019.
- [14] Y. Wang, Y. Huang, W. Zheng, Z. Zhou, D. Liu, and M. Lu, “Combining convolutional neural network and self-adaptive algorithm to defeat synthetic multi-digit text-based captcha,” Technical Report or Unpublished Manuscript, n.d.
- [15] W. Ouyang, X. Wang, X. Zeng, S. Qiu, P. Luo, Y. Tian, H. Li, S. Yang, Z. Wang, C.-C. Loy, and X. Tang, “DeepID-Net: Deformable deep convolutional neural networks for object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2015, pp. 2403–2412.